

CC, SI & IA – 2ª série

Manutenção e Reengenharia de Software

2025

Bibliografia Básica

- PRESSMAN, Roger S; MAXIM, Bruce R. Engenharia de software: uma abordagem profissional. 9. ed. Porto Alegre: AMGH, 2021.
<https://integrada.minhabiblioteca.com.br/#/books/9786558040118>
- SOMMERVILLE, Ian. Engenharia de software. 10. ed. São Paulo: Pearson, 2019.

Aula 12 – Teoria

Bibliografia Complementar

- CALVETTI, Robson. Manutenção e Reengenharia de Software. Material de aula, São Paulo, 2022.

Manutenção de Software



O que é?

» **Manutenção de Software:**

- Não importa qual é o domínio de aplicação, tamanho ou complexidade, o *software* continuará a evoluir com o tempo, pois as mudanças motivam esse processo;
- No âmbito do *software*, alterações ocorrem quando:
 - ✓ Erros são corrigidos;
 - ✓ Quando há adaptação a um novo ambiente, quando;
 - ✓ O cliente solicita novas características ou funções; e
 - ✓ Quando a aplicação passa por um processo de reengenharia, para se atualizar a um contexto moderno.

Quando começa?

» **Manutenção de *Software*:**

Os problemas:

- O *software* é liberado para os usuários e, em alguns dias, os relatos de erros começam a chegar;
- Em algumas semanas, uma classe de usuários indica que o *software* deve ser mudado para se adaptar às necessidades especiais de seus ambientes;
- Em alguns meses, outro grupo corporativo, que não estava interessado no *software* quando ele foi lançado, reconhece que ele pode trazer vantagens, mas reportam que precisarão de algumas melhorias para fazer o *software* funcionar em “seu mundo”.

Como fazer?

» **Manutenção de Software:**

Os desafios:

- Enfrenta-se uma fila cada vez maior de correções de erros, solicitações de adaptação e melhorias que devem ser planejadas, programadas e, por fim, executadas;
- Logo, a fila já cresceu muito, e o trabalho ameaça devorar os recursos disponíveis;
- Com o passar do tempo, a empresa desenvolvedora descobre que está gastando mais tempo e dinheiro ($\approx 60\%$ a 70% dos recursos) com a manutenção dos programas do que criando novas aplicações;

Como fazer?

» **Manutenção de Software:**

Os desafios:

- Quanto à mobilidade dos profissionais, é provável que a equipe ou pessoa responsável pelo trabalho original não esteja mais por perto;
- Pior ainda, outras gerações de profissionais de *software* podem ter modificado o sistema e já se foram, também;
- E hoje pode não ter restado ninguém que tenha conhecimento direto ou indireto do sistema legado.

Conclusões!

» **Manutenção de Software:**

- A manutenção é caracterizada pela modificação do *software* já entregue ao cliente, ou seja, a manutenção é qualquer alteração no *software* após sua entrada em produção.

Quais são?

» **Diferenças entre as Manutenções de *Software*:**

- Embora sua definição trate genericamente qualquer produto de *software*, existem diferenças entre elas com propósitos distintos:
 1. *Softwares* construídos com base em uma especificação rígida e bem definida, cujos resultados esperados são bem conhecidos, *e.g.* um *software* construído para realizar operações com matrizes, do tipo adição, multiplicação e inversão, uma vez que tenha sido construído considerando a correta implementação do método, dificilmente haverá a necessidade de manutenções;

Quais são?

» Diferenças entre as Manutenções de *Software*:

2. *Softwares* que constituem implementações de soluções aproximadas para problemas do mundo real, uma vez que soluções completas somente são conseguidas na teoria nesses casos, *e.g.* num jogo de xadrez, embora suas regras sejam bem definidas, não é possível construir um *software* que calcule rapidamente a cada passo todos os inúmeros possíveis movimentos das peças do tabuleiro para determinar o melhor movimento, sendo que a técnica utilizada para desenvolver esse tipo de solução baseia-se em descrever o problema de forma abstrata e então definir os requisitos de *software* a partir dessa abstração.

Quais são?

» **Diferenças entre as Manutenções de *Software*:**

3. *Softwares* que têm mudanças no ambiente onde vai ser utilizado, característica não existente nas duas classificações anteriores, sendo aquele que é criado com base em um modelo dos processos abstratos envolvidos no sistema e precisará mudar sempre que ocorrer mudanças nesses modelos, sendo, portanto, parte do ambiente que ele modela, e.g. aqueles que apresentam informações da economia de um país, com o modelo mudando conforme a economia vai se estabelecendo e com ele a abstração do problema, causando uma necessidade inevitável de manutenção no *software*.

Aula 12 – Manutenção de Software

Quais são?

» **Manutenções de *Software*:**

- As manutenções não se caracterizam apenas por correções, existindo três tipos principais:
 - i. Manutenções de *Software* Adaptativas;
 - ii. Manutenções de *Software* Corretivas;
 - iii. Manutenções de *Software* Evolutivas.

O que é?

» **Manutenção de *Software* Adaptativa:**

- São alterações que visam adaptar o *software* a uma nova realidade ou novo ambiente externo, normalmente imposto;
- Um exemplo claro seriam mudanças de leis ou regras, definidas pelo governo e/ou órgãos reguladores requerendo adequar o software ao seu ambiente externo;
- Outro exemplo, seria numa atualização de um gerenciador de banco de dados, fazendo parte de um sistema maior de *hardware* e *software*, quando os programadores percebem que as rotinas de acesso a disco existentes precisariam de um parâmetro adicional para atender às novas expectativas de uso.

O que é?

» **Manutenção de *Software* Corretiva:**

- Servem para eliminar as falhas encontradas em produção, bastante comuns, principalmente quando o processo de desenvolvimento não se preocupou de maneira adequada com a qualidade do *software*, visando corrigir defeitos de funcionalidade, o que inclui acertos emergenciais de um programa, e.g. quando um usuário reporta um problema de impressão em um relatório, sendo identificado como uma falha no *driver* da impressora, provocando a necessidade de se alterar o menu do relatório para aceitar um parâmetro adicional que determina o número máximo de linhas impressas por folha.

Aula 12 – Manutenção de Software

O que é?

» **Manutenção de *Software* Evolutiva:**

- São alterações que visam agregar novas funcionalidades e melhorias para os usuários que as solicitaram, não devendo confundir esse tipo de manutenção com as entregas programadas de um processo de desenvolvimento iterativo, *e.g.* uma integração com outros sistemas também é considerada um tipo de evolução.

Quais são?

» **Processos de Manutenção de *Software*:**

- A atividade de manutenção requer o cumprimento de algumas etapas consideradas adequadas para se obter um resultado positivo;
- Deve-se primeiramente avaliar a documentação e o código existentes, juntamente com a arquitetura, estrutura de dados e interface;
- Posteriormente, deve-se identificar as modificações necessárias e avaliar seus impactos. Em seguida, realizar as modificações e testá-las através da aplicação de testes.

Quais são?

» **Problemas na Manutenção de *Software*:**

- Alguns problemas podem ocorrer durante a realização de algum processo de manutenção;
- Devem ser, na medida do possível, previstos na tentativa de serem evitados, tais como:
 - Ausência ou deficiência na documentação;
 - Dificuldade na identificação de manutenções realizadas anteriormente;
 - Falta de controle de versão;
 - Baixa manutenibilidade do *software*.

Aula 12 – Manutenção de Software

Quais são?

» **Efeitos Colaterais da Manutenção de *Software*:**

- Podem ocorrer com a realização desses processos:
 - Modificação da estrutura do código;
 - Inclusão de códigos com erros;
 - Desatualização da documentação.

Aula 12 – Manutenção de Software

Quais são?

» Antídotos na Manutenção de *Software*:

- Deve-se identificar e registrar todas as partes que foram afetadas pela modificação;
- Para melhorar o processo, devem ser levadas em conta as mudanças ocorridas no ambiente em que está inserido o sistema.

Quais são?

» **Fases da Manutenção de *Software*:**

1. Introdução: durante esta fase, existe um grande suporte ao usuário, período em que são concebidas as primeiras ideias sobre o sistema;
2. Crescimento: fase de ajuste, onde o sistema já está concretizado e muitas vezes as correções aparecem naturalmente;
3. Maturidade: fase de ajuste, onde é necessário que sejam realizados testes e atualização da documentação para avaliar as partes modificadas e acrescentadas na aplicação; e

Quais são?

» **Fases da Manutenção de *Software*:**

4. Declínio: fase onde o sistema chega ao final da vida útil e deve ser avaliado se o sistema deve ser retirado de operação.

Essa avaliação deve ser feita com base nos aspectos econômicos, como, por exemplo, substituição de tecnologias ou, até mesmo, para conseguir a independência de um fabricante.

O que é?

» **Manutenibilidade:**

- Também conhecida por Manutenabilidade;
- Tanto a análise quanto o projeto levam a uma importante característica do *software* que se chama manutenibilidade;
- Em essência, manutenibilidade é um indicativo qualitativo da facilidade de corrigir, adaptar ou melhorar o *software*, ou seja de sofrer manutenções;
- Grande parte das funções da Engenharia de *Software* está focada em criar sistemas que apresentem alta manutenibilidade.

Quais são?

» **Características da Manutenibilidade:**

O *Software* manutenível:

- Apresenta uma modularidade eficaz;
- Utiliza padrões de projeto que permitem entendê-lo facilmente;
- Foi construído usando padrões e convenções de codificação bem definidos, levando a um código-fonte auto documentado e inteligível;
- Passou por técnicas de garantia de qualidade que descobriram problemas de manutenção em potencial antes que o software fosse lançado;

Quais são?

» **Características da Manutenibilidade:**

O *Software* manutenível:

- Foi criado por Engenheiros de *Software* que reconhecem que não estarão por perto quando as alterações tiverem de ser feitas;
- Portanto, o projeto e a implementação do *software* devem “ajudar” a pessoa que for fazer a alteração.

O que é?

» **Suportabilidade de *Software*:**

- A fim de fornecer efetivamente suporte a *software* de classe industrial, a empresa, ou seus projetistas, deve ser capaz de fazer correções, adaptações e melhorias inerentes à atividade de manutenção;
- Além disso, a empresa deve desempenhar outras atividades importantes, que incluem suporte operacional continuado, suporte ao usuário e atividades de reengenharia durante toda a vida útil do *software*;
- É a capacidade de fornecer suporte a um sistema de *software* durante toda a vida útil do produto.

Reengenharia de *Software*



Aula 12 – Reengenharia de Software

O que é?

» **Reengenharia de Software:**

- Reconstruir um sistema;
- Melhorar desempenho, manutenibilidade, segurança, documentação etc.;
- Melhorar estruturas e entendimento;
- Sistemas legados:
 - ✓ Muito valioso;
 - ✓ Linguagens de programação antigas: COBOL, PHP etc.;
 - ✓ Tecnologia obsoleta;
 - ✓ Sofreu muitas mudanças, dando sinais de adaptações;
 - ✓ Não pode ser descartado e não deve ser atualizado, por ser muito custoso e arriscado.

Aula 12 – Reengenharia de Software

Quais são?

» **Benefícios da Reengenharia de *Software*:**

- Reduzir a complexidade de sistemas legados:
 - Portar para uma nova plataforma;
 - Melhorar performance;
 - Extrair modelos;
 - Explorar novas tecnologias;
 - Reduzir dependência humana.

Quais são?

» **Benefícios da Reengenharia de Software:**

- Reduzir riscos:
 - Risco de desenvolver o sistema novamente é alto;
 - Podem ocorrer erros de especificação ou problemas no desenvolvimento.
- Reduzir custos:
 - Custo de reengenharia pode ser muito menor que o custo de desenvolvimento (4x);
 - Ferramentas podem automatizar algumas partes.

Quais são?

» **Benefícios da Reengenharia de *Software*:**

- Quando aplicar:
 - Documentação obsoleta ou não existente;
 - Desenvolvedores deixaram a empresa;
 - Conhecimento limitado do sistema;
 - Tempo elevado para realizar pequenas mudanças;
 - Frequente correção de defeitos;
 - Problemas de manutenção (efeito cascata).

Quais são?

» **Comparativos entre Engenharia Vs. Reengenharia:**

- Engenharia Reversa:

Processo de analisar um sistema para identificar os componentes e seus inter-relacionamentos e criar representações em um alto grau de abstração, cuja palavra chave é “ENTENDER”;

- Reengenharia:

É o exame e a alteração de um sistema para reconstituí-lo e em seguida implementá-lo de uma nova forma, cuja palavra chave é “REESTRUTURAR”.

Quais são?

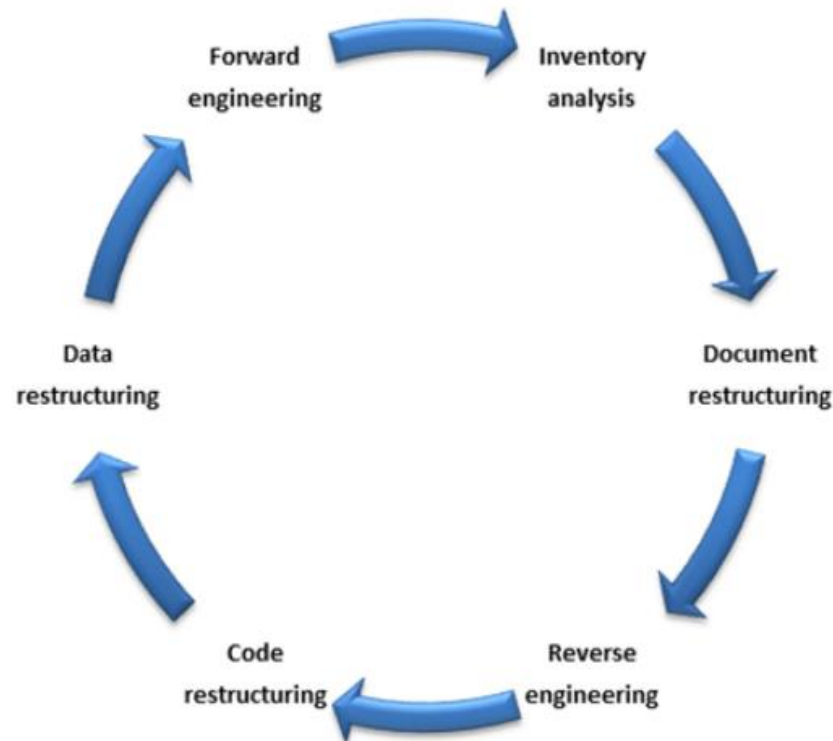
» Comparativos entre Engenharia Vs. Reengenharia:



Como é?

» Processo de Reengenharia de Software:

- ✓ Etapas bem definidas;
- ✓ Cíclico.



Aula 12 – Reengenharia de Software

Como é?

» Processo de Reengenharia de *Software*:

1. Análise de Inventário:

- Toda organização tem um inventário de aplicações;
- Organizado em uma planilha por tamanho, idade, importância etc.;
- Priorizar sistemas candidatos;
- Alocar recursos.

Aula 12 – Reengenharia de Software

Como é?

» Processo de Reengenharia de *Software*:

2. Reestruturação de Documentação:

- Documentação fraca e escassa é a marca de sistema legados;
- Opções:
 - Nada fazer;
 - Documentar apenas mudanças;
 - Documentar o mínimo necessário.

Aula 12 – Reengenharia de Software

Como é?

» Processo de Reengenharia de *Software*:

3. Engenharia Reversa (1):

- Processo de extrair informação a partir do código;
Exemplo: extrair os diagramas de classe;
- Informação extraída pode ser provida para:
 - Engenheiro de *software*; e
 - Ferramenta.

Aula 12 – Reengenharia de Software

Como é?

» Processo de Reengenharia de *Software*:

3. Engenharia Reversa (2):

- Entender:
 - Dados internos e globais;
 - Processamento de abstrações procedurais, com ferramentas;
 - Interface com o usuário, com estrutura e comportamento da *UI*.

Aula 12 – Reengenharia de Software

Como é?

» **Processo de Reengenharia de Software:**

4. Restruturação de Código:

- Modifica código;
- Não altera a arquitetura;
- Código com mesmas funcionalidades, porém, com mais qualidade.

5. Restruturação de Dados:

- Modifica dados;
- Não altera a arquitetura;
- Projeto de dados mais efetivo, e.g. mudando o BD;

Aula 12 – Reengenharia de Software

Como é?

» **Processo de Reengenharia de *Software*:**

6. Engenharia Avante:

- Recupera o projeto de *software*;
- Melhora a qualidade do sistema;
- Adiciona novas funcionalidades
- Modifica a arquitetura;
- Promove uma nova configuração do sistema.

O que é?

» Engenharia Reversa:

- Imagine:
 - ✓ Coloca-se em uma máquina os códigos-fontes de um projeto não documentado, criada de qualquer jeito;
 - ✓ Aperta-se um botão na máquina;
 - ✓ Do outro lado, sai da máquina uma documentação completa, com descrições detalhadas do projeto.
- Infelizmente, essa máquina não existe;

O que é?

» Engenharia Reversa:

- A engenharia reversa pode extrair informações do projeto a partir do código-fonte, mas o nível de abstração, a completude da documentação, o grau, segundo o qual as ferramentas e um analista humano trabalham juntos e a direcionalidade do processo são altamente variáveis.

O que é?

» **Nível de Abstração da Engenharia Reversa:**

- O processo de engenharia reversa deve ser capaz de derivar representações de projeto procedural (em uma abstração de baixo nível), informações de programa e de estrutura de dados (em um nível de abstração um tanto mais alto), modelos de objeto, dados e/ou modelos de fluxo de controle (em um nível de abstração relativamente alto) e modelos de dados (em um nível alto de abstração);
- Conforme o nível de abstração aumenta, recebem-se informações que permitirão entender o programa mais facilmente.

O que é?

» **Completude da Engenharia Reversa:**

- A completude de um processo de engenharia reversa se refere ao nível de detalhe fornecido em um nível de abstração;
- Em muitos casos, a completude diminui conforme o nível de abstração aumenta;
- Por exemplo, dada uma listagem de código-fonte, é relativamente fácil desenvolver uma representação completa do projeto procedural;

O que é?

» **Completude da Engenharia Reversa:**

- Podem ser derivadas também representações simples do projeto arquitetural, mas é muito mais difícil desenvolver um conjunto completo de diagramas UML ou modelos;
- A completude melhora em proporção direta com a quantidade de análise realizada pela pessoa que está fazendo a engenharia reversa.

O que é?

» **Interatividade da Engenharia Reversa:**

- A interatividade refere-se ao grau segundo o qual as pessoas são “integradas” às ferramentas automáticas para criar um processo eficaz de engenharia reversa;
- Em muitos casos, à medida que o nível de abstração aumenta, a interatividade deve aumentar para que a completude não seja prejudicada.

O que é?

» **Direcionalidade da Engenharia Reversa:**

- Se a direcionalidade do processo de engenharia reversa for unidirecional, todas as informações extraídas do código-fonte serão fornecidas ao engenheiro de software, que pode então usá-las durante qualquer atividade de manutenção;
- Se a direcionalidade for bidirecional, as informações serão colocadas em uma ferramenta de reengenharia que tenta reestruturar ou regenerar o sistema antigo;
- Antes de começar as atividades de engenharia reversa, o código-fonte não estruturado (“ruim”) é reestruturado para que contenha somente as construções de programação estruturada;

O que é?

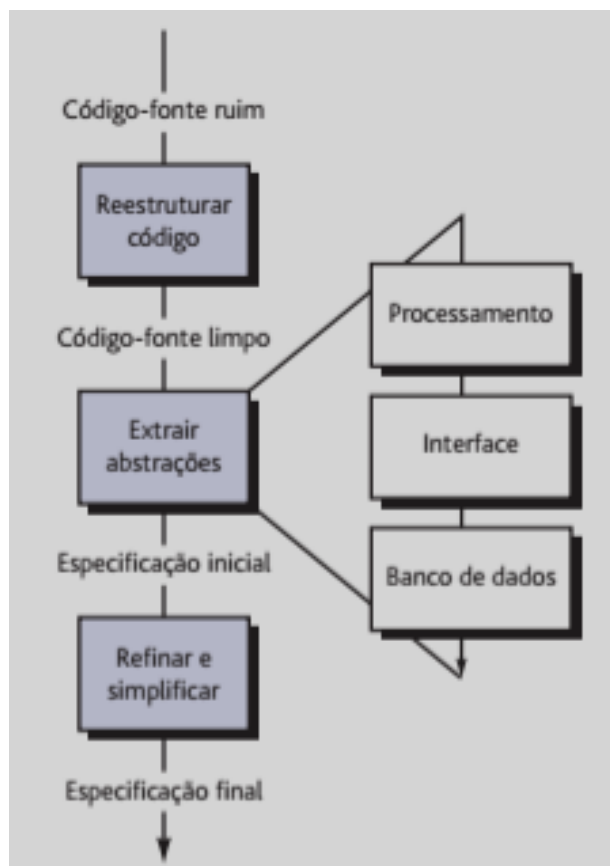
» **Direcionalidade da Engenharia Reversa:**

- Isso torna o código-fonte mais fácil de ler e proporciona a base para todas as atividades subsequentes de engenharia reversa;
- O centro da engenharia reversa é uma atividade chamada extrair abstrações;
- Deve-se avaliar o programa antigo e, com base no código-fonte (muitas vezes não documentado), desenvolver uma especificação significativa do processamento executado, da interface de usuário aplicada e das estruturas de dados de programa ou do banco de dados utilizadas.

Aula 12 – Reengenharia de Software

Qual é?

» Processo de Engenharia Reversa:



O que é?

» **Reestruturação de *Software*:**

- Modifica o código-fonte e/ou os dados para torná-lo mais amigável para futuras alterações;
- Em geral, não modifica a arquitetura geral do programa;
- Tende a se concentrar nos detalhes de projeto dos módulos e nas estruturas de dados locais definidas nos módulos;
- Se a reestruturação vai além dos limites dos módulos e abrange a arquitetura do software, ela passa a ser engenharia direta;

O que é?

» **Reestruturação de *Software*:**

- Ocorre quando a arquitetura básica de uma aplicação é sólida, mesmo que as partes técnicas internas necessitem de retrabalho;
- Ocorre quando partes importantes do *software* são reparáveis e somente um subconjunto de todos os módulos e dados necessita de uma modificação mais extensa.

O que é?

» **Reestruturação de Código:**

- É feita para gerar um projeto que produz a mesma função, mas com mais qualidade do que o programa original;
- Em geral, as técnicas de reestruturação de código, por exemplo, as técnicas de simplificação lógica de Warnier, modelam da lógica de programação usando álgebra booleana e aplicam uma série de regras de transformação que resulta em lógica reestruturada;
- O objetivo é pegar um código “emaranhado” e dele derivar um projeto procedural que esteja em conformidade com a filosofia de programação estruturada;

O que é?

» **Reestruturação de Código:**

- Outras técnicas de reestruturação também foram propostas para uso com as ferramentas de reengenharia;
- Um diagrama de intercâmbio de recursos mapeia cada módulo de programa e os recursos (tipos de dados, procedimentos e variáveis) permutados entre ele e outros módulos;
- Criando representações do fluxo de recursos, a arquitetura do programa pode ser reestruturada para obter o mínimo acoplamento entre os módulos.

O que é?

» **Reestruturação de Dados:**

- Antes de iniciar a reestruturação de dados, deve ser feita uma atividade de engenharia reversa chamada análise do código-fonte;
- São avaliadas todas as instruções da linguagem de programação que contenham definições de dados, descrições de arquivo, E/S (I/O) e descrições de interface;

O que é?

» **Reestruturação de Dados:**

- A finalidade é extrair itens de dados e objetos para obter informações sobre fluxo de dados e para entender as estruturas de dados existentes que precisam ser implementadas;
- Essa atividade às vezes é chamada de análise de dados;
- Uma vez concluída a análise de dados, inicia-se o reprojeto dos dados;

O que é?

» **Reestruturação de Dados:**

- Em sua forma mais simples, uma etapa de padronização de registro de dados esclarece as definições de dados para obter consistência entre nomes de itens de dados ou formatos de registros físicos em uma estrutura de dados ou formato de arquivo existente;
- Outra forma de reprojeto, chamada de racionalização de nomes de dados, garante que todas as convenções de nomes de dados estejam de acordo com os padrões locais e que os pseudônimos (*aliases*) sejam eliminados à medida que os dados fluem pelo sistema.

O que é?

» **Reestruturação de Dados:**

- Quando a reestruturação vai além da padronização e da racionalização, são feitas modificações físicas nas estruturas de dados existentes para tornar o projeto de dados mais eficaz;
- Isso pode significar a transformação de um formato de arquivo em outro ou, em alguns casos, a transformação de um tipo de banco de dados em outro.

Aula 12 – Reengenharia de Software

O que é?

» Engenharia Direta:

- Por exemplo, um programa com um fluxo de controle linear, que equivale graficamente a um prato de espaguete, com módulos de 2 mil instruções, algumas poucas linhas de comentários úteis em 290 mil instruções de código-fonte e nenhuma outra documentação, deve ser modificado para acomodar alterações e requisitos de usuário;

Quais são?

» **Opções da Engenharia Direta:**

1. Pode-se trabalhar arduamente, fazendo modificação após modificação, enfrentando problemas de projeto improvisado e de código-fonte confuso para implementar as mudanças necessárias;
2. Pode-se tentar entender os detalhes internos do programa, em um esforço para tornar as modificações mais eficazes;
3. Pode-se reprojeter, recodificar e testar as partes do software que exigem modificação, aplicando uma abordagem de engenharia de software a todos os segmentos revisados;

Quais são?

» **Opções da Engenharia Direta:**

4. Pode-se reprojeter completamente, recodificar e testar o programa, usando ferramentas de reengenharia para nos ajudar a entender o projeto atual.
- Não há uma opção “correta”, pois as circunstâncias podem recomendar uma das opções, mesmo que as outras sejam mais desejáveis.

Quais são?

» Aspectos Econômicos da Engenharia Direta:

- Em um mundo perfeito, todo programa não manutenível seria aposentado imediatamente para ser substituído por aplicações de alta qualidade, desenvolvidas com modernas práticas de engenharia de software;
- Mas o mundo real tem sempre recursos limitados;
- A reengenharia consome recursos que podem ser utilizados para outras finalidades de negócio;
- Portanto, antes de se pensar em fazer a reengenharia de uma aplicação existente, uma empresa deve fazer uma análise de custo-benefício.

Conclusões!

» Reengenharia:

- A finalidade dessas atividades é criar versões dos programas existentes que tenham melhor qualidade e melhor manutenibilidade, programas que serão viáveis ao longo do século 21;
- O custo-benefício da reengenharia pode ser determinado quantitativamente;
- O custo do estado atual, isto é, o custo associado ao suporte e à manutenção continuados de uma aplicação existente é comparado aos custos projetados da reengenharia e à redução resultante nos custos de manutenção e suporte;

Conclusões!

» **Reengenharia:**

- Em quase todos os casos nos quais um programa tem uma vida longa e em certo momento apresenta manutenibilidade ou suportabilidade ruim, a reengenharia representa uma estratégia de negócio eficiente em termos de custo.

Aula 12 – Teoria

FIM